

BT4012: Fraud Analytics

Kaggle Competition 2026

Detecting Fraudulent Bitcoin Transactions

Melvin Loo
A0282157B

Code: github.com/melvinloo091/bt4012-bitcoin-fraud-detection

1. Introduction

Bitcoin lets anyone send money without a bank in the middle, which also attracts crime such as scams, ransomware and dark-web markets. Investigators cannot check every payment by hand, so we need a model that sorts transactions from most to least suspicious and shows them where to look first. This competition asks for exactly that: a *suspicion score* for every transaction, so that *illicit* (crime-linked) transactions score higher than *licit* (legitimate) ones.

The data. Transactions are grouped into 49 *time steps*. Our models learn from steps 1–35 (31,235 transactions with known labels, 12% of them illicit, plus 110,537 with unknown labels) and score 15,329 transactions from steps 36–49. Each transaction has 165 anonymised numbers called *features*: features 1–93 describe the transaction itself, and features 94–165 summarise its *neighbours*, the transactions it is directly linked to. Finally, 234,355 "who paid whom" links join the transactions into a network, which data scientists call a *graph*.

Scoring. Kaggle uses *ROC-AUC*: the chance that a randomly picked illicit transaction scores higher than a randomly picked licit one (0.5 = coin toss, 1.0 = perfect). Plain accuracy would mislead here, because calling every transaction licit is already 88% accurate on the training data. During the competition, Kaggle shows a *public score* based on 30% of the test rows; the final ranking uses the hidden other 70%.

The main difficulty. A large dark market shut down at step 43, inside the test period, after which the illicit share falls from about 11% to under 2%. It is like revising with past years' papers when the syllabus changes halfway through the period.

Approach and outcome. My goal was a model that ranks *future* transactions well, rather than one that memorises the past. My key strategies were to test every idea on later time steps than it learned from, to average two tree-based models and to give recent transactions more weight. The final model raised my public score from 0.93654 to **0.95527** (course baseline is about 0.92).

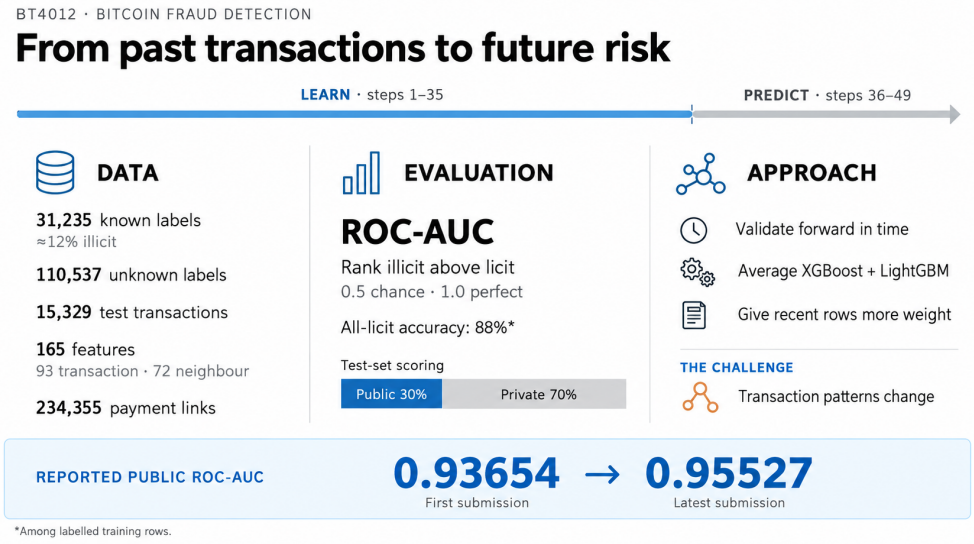


Figure 1. Introduction Summary

2. Methodology

2.1 Data Preprocessing and Feature Engineering

Missing values. No feature values are missing, and the features already arrive on a common scale, so little cleaning was needed. What is missing are the labels. I tried four ways to use the unknown rows: (1) treating them all as licit; (2) pseudo-labelling (letting a model guess their labels, then learning from its own guesses); (3) copying labels to linked neighbours; and (4) an autoencoder (a neural network that learns a short summary of every transaction, used as extra features). All four lowered my local scores, since wrong guesses teach wrong answers, therefore the final model learns from labelled rows only.

Feature selection. Six features (100, 101, 103, 136, 137 and 139) rise almost in step with time: sorting transactions by any of them nearly sorts them by time step (rank correlation > 0.9). The share of illicit transactions swings a lot over time, from 0.4% in some steps to 36% in others. These calendar features invite rules about when a transaction happened rather than how it behaves. Removing them from XGBoost (Section 2.3) raised my hardest local test results from 0.915 to 0.921.

Graph-based features. I tested common network measures: (1) node degree (how many edges each transaction has), (2) triangles and clustering (whether its neighbours are related to each other) and (3) PageRank (how central it is in the network). These measures lowered my back-test average (Section 2.2) by about 0.005, and graph neural networks, which pass information along the edges, lost on every test. Two reasons could explain: features 94–165 already summarise each transaction's neighbours, and two-thirds of the edges around test transactions led to transactions missing from the files. What did help were nine simple counts, given to XGBoost only: payments in, payments out and number of partners; transactions two links away; and five facts about the partners (their average and largest number of links, their average payments out and in, and the share of partners that simply pass money along, with one payment in and one payment out). The improvement tied on my practice exams but raised the public score to 0.95524.

Labelled-neighbour counts. The missing transactions turned out to be exactly the 46,668 unlabelled ones from steps 36–49, so each transaction's labelled partners can be counted the same way in training and test data. This looked useful: transactions with no labelled partner were illicit 24% of the time, against 8% for those with at least one labelled partner. But adding these counts did not improve the model (practice exams unchanged; Kaggle 0.95499), probably because features 94–165 already capture the same information. So the final model does not use them.

2.2 Model Development and Validation

Splitting by time. Cross-validation, a common check, hides a random slice of the data, trains on the rest and tests on the hidden slice. This would be misleading, because transactions from the same time step are closely related. Suppose a fraud ring is active in step 20: a random split puts half of its transactions in training and half in the test slice, so the model only has to recognise a ring it has already seen. So I always deliberately trained on earlier steps and tested on later ones. Even so, my first model scored 0.995 on steps 29–35 but only 0.937 on Kaggle.

Yardstick: 15 "practice exams". To judge each change before uploading it, I used a time-based form of cross-validation: each practice exam learns from steps 1 to k and tests on the next 14 steps (k = 7 to 21), and my local score is the average ROC-AUC over the 15 exams. This yardstick drove most of my training decisions: dropping the calendar features, choosing gradient boosting over the other model types, lightening the brakes and leaving the classes unweighted.

Blind spots and a safety check. The exams contain no shutdown, so they cannot judge changes aimed at the post-shutdown period. That is why the recency strength was chosen on Kaggle instead (Section 3). They are also fooled by methods that compare test rows with each other: a "look-alike" score built on near-duplicate transactions reached 0.9713 locally but 0.87805 on Kaggle. Before each upload, I therefore checked that the new model ranked test transactions in roughly the same order as my best model, and that its most suspicious rows were not ones my best model considered safe.

Hyperparameters and overfitting. Hyperparameters are the settings chosen before training, such as how big each tree may grow. Overfitting means memorising noise in the training data, like a student who memorises past answers instead of learning the method. Some settings, called regularisation, act like brakes: too little and the model memorises noise, too much and it misses small fraud rings. Lighter brakes beat my early heavy ones in 13 of 15 exams, while small trees (at most 15 leaves, the end points where a tree gives its answer) and a slow learning rate (0.05, each new tree's correction is scaled down to 5%, so no single tree can swing the scores much) still guard against overfitting.

Class imbalance. Only 12% of labelled rows are illicit. A common fix is to give them extra weight, but ROC-AUC only judges the order of the scores. Weighting illicit rows 7.6x lowered the practice-exam average by 0.009, so the final model trains on the classes as they are; its only weighting is the recency weighting in Section 2.3.

2.3 Model Exploration and Final Strategy

Models explored. I compared each model type on the same local tests. Most are built from decision trees: flowcharts of yes/no questions about the features (e.g., "is feature 5 above 0.3?") that end in a score.

Figure 2. Model Exploration: How they work and the results

Model	How it works	Result
Logistic regression	A weighted checklist: each feature adds or subtracts points	Too simple: 0.893
Random forest, Extra Trees	Hundreds of independent trees, each built separately, that vote	Weak in hard periods (Extra Trees: 0.917 on Kaggle)
Neural networks, including graph neural networks	Layers of simple maths units learn patterns; the graph versions also pass information along the payment links	Lost on every practice exam

Model	How it works	Result
Gradient boosting	Small trees built one after another, like a relay team in which each new runner fixes the mistakes the team has made so far	**Won every comparison**
HGB	scikit-learn's gradient boosting; my first boosted model	0.931 locally, 0.937 on Kaggle
CatBoost	Gradient boosting designed for text-like categories	0.919; adding it to the average also lowered the score
XGBoost	"eXtreme Gradient Boosting": the classic, widely used version, with strong built-in brakes	**Used in the final model**
LightGBM	Microsoft's "Light Gradient Boosting Machine": built for speed; always grows the branch that fixes the most error first	**Used in the final model**

Final strategy. The final model has three ingredients:

1. **Two models averaged. XGBoost** and **LightGBM**, averaged 50/50. The two models are built differently, so they make slightly different mistakes, and averaging their scores lets those mistakes partly cancel out. They also see different inputs: XGBoost (300 trees) drops the six calendar features and adds the nine network counts, while LightGBM (150 trees) uses all 165 features.
2. **DART (Dropouts meet Multiple Additive Regression Trees) training for XGBoost.** In normal boosting, the first few trees do most of the work and later trees only polish tiny leftover errors, which can mean learning noise. DART borrows the "dropout" trick from neural networks: it randomly switches off some earlier trees while each new tree is built, so every new tree must be useful on its own and no early tree dominates. The price is speed: about 9 minutes of training instead of a few seconds.
3. **Recency weighting.** Each training transaction counts $0.5^{(35-\text{step})/4}$ times. With this half-life of 4, step 31 counts half as much as step 35 and step 27 a quarter as much. I chose the half-life using Kaggle submissions.

Why this model. My two Kaggle selections are (1) this model and (2) the same model with ordinary XGBoost without DART. They are effectively tied (0.9498 vs 0.9505 on the local tests; 0.95527 vs 0.95524 on Kaggle), and DART is my final model for its slightly higher public score.

The notebook e1156285.ipynb in my GitHub repository (github.com/melvinloo091/bt4012-bitcoin-fraud-detection) rebuilds the 0.95527 file exactly.

3. Results and Discussion

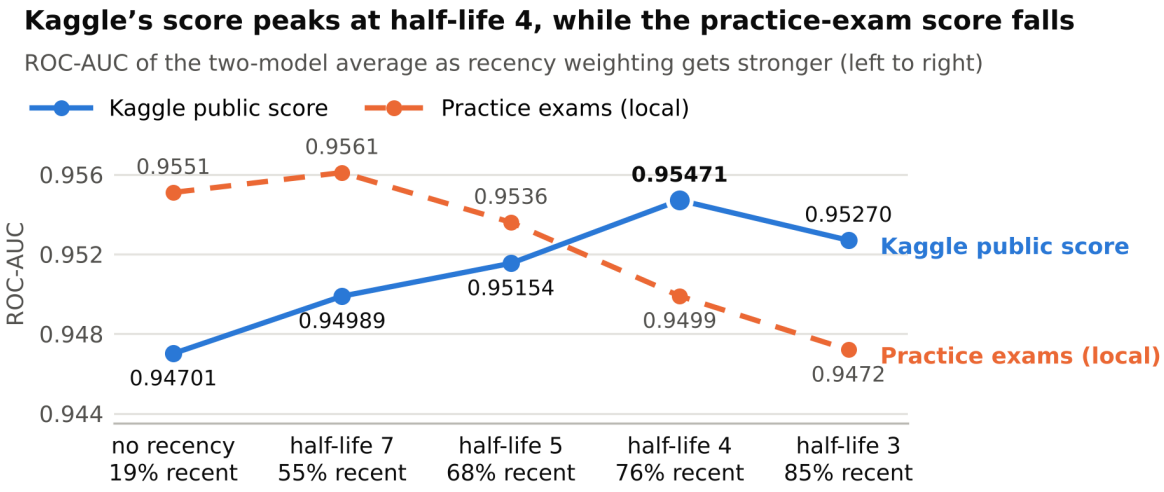


Figure 3. Kaggle public scores and the 15-exam practice (local tests) average;

"% recent" is the share of training weight on steps 29–35.

Recency weighting gave the largest gain: +0.0077 on Kaggle from no recency to a half-life of 4. Going further, to 3, lowered the score, because stronger recency also throws data away: at a half-life of 4, the 31,235 labelled transactions carry the weight of only about 8,800 equally weighted ones. The local tests moved the other way (0.9551 down to 0.9472), and this is not a contradiction. They come from steps 1–35, before the shutdown, when behaviour was fairly stable, so old data is as useful as new data and fading it only costs examples. The real test

period contains the shutdown, where recent behaviour matters more, and only Kaggle's test covers it. I expended some submission onto Kaggle to determine that half-life 4 edges 5 by 0.003, so part of that gain may also be luck.

Figure 4. Key submissions. Each adds one change to the row above; the look-alike blend was dropped.

Submission	Change	Local Tests	Kaggle public
Histogram gradient boosting	First boosted trees	0.9310	0.93654
Tuned XGBoost	Small trees, heavy brakes	0.9409	0.94265
Look-alike blend	Near-duplicate score	0.9713	0.87805
Two-model average	Light brakes, + LightGBM	0.9551	0.94701
Recency weighting	Half-life 4	0.9499	0.95471
More trees	300 XGBoost trees	0.9507	0.95484
Network counts	+ 9 network counts	0.9505	0.95524
Final model	**XGBoost with DART**	**0.9498**	**0.95527**

Reading Figure 4. Three steps did most of the work: XGBoost with small trees (+0.006 over the first model), lighter brakes plus averaging with LightGBM (+0.004) and recency weighting (+0.008). The other changes added less than 0.0005. The look-alike row is a warning: a high local score can hide a poor Kaggle score.

Why the score stopped rising. After the 300 XGBoost trees model, none of my 13 further models beat 0.95484 by more than 0.0005, and bigger changes often scored lower: dropping the features that change most over time gained 0.002 locally but scored 0.94474 on Kaggle. The public score uses only 30% of the test rows and may differ from the hidden 70% by about ±0.007–0.009 by chance alone. Chasing gains far smaller than that would have meant overfitting the public tests, tuning to the quirks of those 30% of rows rather than improving the model, so I treated them as ties and stopped tuning.

Strengths and limitations. The model is simple and exactly reproducible, and every choice was tested on later time periods than it learned from. Its main weakness is **new fraud rings**: in the hardest local tests, 96–98% of the missed illicit transactions were linked to other missed ones, so when a ring unlike anything in training appears, the model tends to miss the whole ring. And because the features are anonymised, the model cannot explain its flags in plain words: saying a transaction scored high because of features 47 and 112 means nothing to an investigator. In practice, each flag should come with reasons, such as which network counts pushed its score up, so that it can be checked and acted on.

4. Conclusion and Reflection

My final model raised the public ROC-AUC from an initial 0.93654 to 0.95527. What worked was simple: testing forward in time, averaging two boosted-tree models and giving recent data more weight. Three lessons stand out:

1. **A practice test must resemble the real exam:** one easy period promised 0.995, and even 15 forward-in-time tests could not predict the shutdown.
2. **Complex is not automatically better:** graph neural networks, autoencoders and the look-alike score all lost to plain trees.
3. **Do not over-read small differences:** on a public leaderboard, gaps below about 0.005 may simply be luck.

For AI-driven fraud detection, the main lesson is that the target keeps moving: the shutdown changed the data inside the test period, and new fraud rings look unlike past ones. A model used in practice would need regular retraining on recent data and testing on later periods. Subsequently, we could look for ways to flag fraud rings unlike any in training, for example tight groups of linked transactions that look unusual together, so that investigators can review them.

Acknowledgement of AI assistance. I used two AI assistants. OpenAI Codex (via ChatGPT) set up the repository and Jupyter notebook and ran the early experiments. Anthropic's Claude proposed and coded the local tests and safety check, wrote and ran most later experiments, helped built the final notebook and drafted this report. I set the goals, chose which models and candidates to upload, compared the two assistants' results and chose the final submissions. All experiments used the competition data only.